



Security Assessment Final Report



Etherfi – Security Hardening

June 2026

Prepared for EtherFi

Table of Contents

Project Summary.....	4
Project Scope.....	4
Project Overview.....	5
Protocol Overview.....	7
Assessment Methodology.....	9
Threat and Security Overview.....	10
Assets.....	10
Actors.....	10
Trust Assumptions.....	11
Attack Vectors.....	11
Findings Summary.....	13
Severity Matrix.....	13
Detailed Findings.....	14
Low Severity Issues.....	18
L-01: Priority withdrawal claims can be bricked for smart-contract recipients that forward or consume received ETH.....	18
L-02: Unpublishing an oracle report can permanently block resubmission for the same slot range.....	19
L-03: Unjustified Reverts in quoteByMarketValue Due to Price Clamping When stETH Trades at a Premium.....	20
L-04: Operator blacklist bypass via missing msg.sender validation in WithdrawRequestNFT.....	21
L-05: Invalid requests cannot be seized from blacklisted users.....	22
L-06: Claims can manipulate finalization rates.....	23
L-07: Invalid requests causing DOS of finalizeWithdrawalsWhenStale.....	25
Informational Severity Issues.....	26
I-01: Fast role revocation can be disabled by an immutable misconfigured revoke admin.....	26
I-02: EtherFiRedemptionManager permanently strands fractional eETH shares.....	27
I-03: cancelWithdraw racing condition.....	28
I-04: Dead parameters in initialize in multiple contracts.....	29
I-05: Oracle Report DoS via invalidateRequest During Reporting Window.....	30
I-06: Unhandled Stale Withdrawal Finalization Causes DoS of Oracle.....	31
I-07: PriorityWithdrawalQueue claiming could abuse the mint/burn rate limits.....	32
I-08: Finalized withdrawals may exceed the liquidity.....	33
I-09: Users in the priority withdrawal queue can earn rewards, without contributing.....	34
I-10: The consumed burn amount may be inflated.....	36
I-11: An attacker could delay any withdrawal request.....	38
I-12: Oracle reports can become impossible to unpublish after quorum or committee changes.....	39
I-13: Blacklisting the zero address can cause protocol-wide mint/burn and withdrawal-claim denial of service.....	41

I-14: Temporary DOS on eETH minting due to rate limiter bucket creation gap.....	42
I-15: EtherFiRestaker setPauseUntilDuration missing from operating config batch.....	43
I-16: Pre-upgrade remainder eETH shares in PWQ and WRN not swept before upgrade.....	44
I-17: I1SyncPoolImpl bytecode not verified against local compilation.....	45
I-18: OZ paused state silently lost across upgrade — contracts unpause regardless of pre-upgrade state.....	46
I-19: `RoleRegistry` excluded from verifyAccessControlPreservation checks.....	47
I-20: ETH swept from AuctionManager to MembershipManager may become inaccessible post-upgrade	48
I-21: Oracle reports straddling the upgrade window cannot be finalized post-upgrade.....	49
I-22: Liquifier/RedemptionManager stETH price feed heartbeat mismatch causes stale price reverts...	50
I-23: DMIN_MAX_REQUESTS_TO_FINALIZE_PER_REPORT = 5000 would consume excessive amount of gas.....	51
I-24: EtherfiL1SyncPoolETH.setPauseUntilDuration not seeded in Batch 2 operating config.....	52
I-25: Halted claims due to request limit being higher than the burn limit.....	53
I-26: Invalidation of blacklisted users' priority withdrawal requests permanently locks ETH.....	54
Disclaimer.....	55
About Certora.....	55

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Latest Commit Hash	Platform
Etherfi - Security Hardening	https://github.com/etherfi-protocol/smart-contracts/pull/385	f169d868c6609682b7733a76faacae33701509dc	ea300389139f64969f9c79e107359c661bf25e54	Solidity
	https://github.com/etherfi-protocol/weETH-cross-chain/pull/77	6554ce0bd8145d6ebe9d6a55dfa9450d7c1f6724	c6997994314a62143a923fdd61c9a11b068d8092	
	https://github.com/etherfi-protocol/avs-smart-contracts/pull/29	c40319be0ede648041130376608b39fca11c6ecd	c40319be0ede648041130376608b39fca11c6ecd	
Scripts Review	https://github.com/etherfi-protocol/smart-contracts/pull/420	b67296629a2adb16caa2a5119c83f156f993417	c8206ca3ba6ae2a93c00aff7ace38b50b93051ea	Solidity

Note: The final deploy contracts are based from commit 0x0f1e8d966c85bcd3b352d94da15fe2eae61f897c (merging #420 -> #385) which is identical to the latest audited hash c8206ca3ba6ae2a93c00aff7ace38b50b93051ea from PR#420

Project Overview

This document describes the security review of **Etherfi – Security Hardening**. The work was undertaken from **May 27th, 2026** to **June 25th, 2026**.

The following contract list is included in our scope:

smart-contracts:

src/core/EETH.sol

src/core/LiquidityPool.sol

src/core/WeETH.sol

src/core/interfaces/ILiquidityPool.sol

src/core/interfaces/IWeETH.sol

src/core/interfaces/IeETH.sol

src/deposits/DepositAdapter.sol

src/deposits/Liquifier.sol

src/governance/Blacklister.sol

src/governance/RevokeAdmin.sol

src/governance/RoleRegistry.sol

src/governance/interfaces/IBlacklister.sol

src/governance/interfaces/IRevokeAdmin.sol

src/governance/interfaces/IRoleRegistry.sol
src/governance/rate-limiting/EtherFiRateLimiter.sol
src/governance/rate-limiting/RateLimitedToken.sol
src/governance/utils/PausableUntil.sol
src/governance/utils/ReentrancyGuardNamespaced.sol
src/governance/utils/RolesLibrary.sol
src/membership/MembershipManager.sol
src/membership/MembershipNFT.sol
src/membership/interfaces/IMembershipManager.sol
src/membership/interfaces/IMembershipNFT.sol
src/oracle/EtherFiAdmin.sol
src/oracle/EtherFiOracle.sol
src/restaking/EtherFiRestaker.sol
src/restaking/RestakingRewardsRouter.sol
src/rewards/CumulativeMerkleRewardsDistributor.sol
src/rewards/EtherFiRewardsRouter.sol
src/staking/AuctionManager.sol
src/staking/EtherFiNode.sol
src/staking/EtherFiNodesManager.sol
src/staking/NodeOperatorManager.sol
src/staking/StakingManager.sol
src/withdrawals/EtherFiRedemptionManager.sol
src/withdrawals/PriorityWithdrawalQueue.sol
src/withdrawals/WeETHWithdrawAdapter.sol



src/withdrawals/WithdrawRequestNFT.sol

avs-smart-contracts:

src/AvsOperatorManager.sol

weETH-cross-chain

contracts/native-minting/EtherfiL1SyncPoolETH.sol

contracts/PausableUntil.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

scripts-review

script/upgrades/security-upgrades/Constants.s.sol

script/upgrades/security-upgrades/deploy.s.sol

script/upgrades/security-upgrades/transactions.s.sol

script/deloys/Deployed.s.sol

Protocol Overview

EtherFi is a liquid staking protocol where users deposit ETH into LiquidityPool and receive eETH (a rebasing ERC-20) proportional to shares. eETH can be wrapped 1:1 into weETH for DeFi compatibility. The protocol stakes deposited ETH into Ethereum validators through a StakingManager / EtherFiNodesManager / EigenPod pipeline, and restakes stETH through EigenLayer via EtherFiRestaker. Withdrawals have three paths: instant redemption via EtherFiRedemptionManager (with a rate-limiter and exit fee), standard queued withdrawal via WithdrawRequestNFT (oracle-finalized, ETH escrowed at finalization), and priority withdrawal via PriorityWithdrawalQueue (whitelisted users, operator-fulfilled).

This upgrade introduces a coordinated security upgrade across six contracts. The headline changes are: (1) a token-level rate limiter (BucketLimiter) on eETH mint, burn, enforced through EtherFiRateLimiter; (2) a dual-layer pause system — an ordinary **paused** boolean plus a timed **PausableUntil** that any Guardian can trigger, with a cooldown; (3) a Blacklist that blocks addresses from minting, burning, transferring, depositing, and withdrawing; (4) a share-rate-freeze upgrade on WithdrawRequestNFT that snapshots the eETH/ETH exchange rate at finalization time so post-finalize rebases cannot change claim amounts; (5) a new **withdraw(uint256 _amount, uint256 _share)** entry point in LiquidityPool with three defensive guards for NFT/queue callers; (6) a **nonDecreasingRate** modifier on LP deposit/burn paths; and (7) a post-upgrade escrow migration that physically moves locked ETH out of LP into the NFT and queue contracts.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Assets

- **Base Tokens:** ETH, stETH.
- **Protocol Tokens:** eETH (rebasing liquid staking token), weETH (non-rebasing wrapped equivalent), and L2ETH (Layer-2 placeholder tokens).
- **Non-Fungible Tokens:** WithdrawRequestNFT (queued withdrawal requests) and MembershipNFT (legacy and current user membership stakes).
- **Abstract Value:** eETH shares, accrued yield and rewards, total value locked (TVL) accounting states, rate-limiter bucket capacities, and oracle consensus data.

Actors

- **Users:** Depositors, liquidity providers, and individuals executing withdrawals.
- **DeFi Integrators & Liquidators:** External smart contracts, protocols, and keeper bots that utilize the protocol's liquidity and token transfers.
- **MEV Searchers / Arbitrageurs:** Network participants monitoring the mempool to front-run state changes or extract just-in-time yield.
- **Upgrade Administrators (Trusted):** Entities controlling proxy upgrades and core system migrations behind a time delay.
- **Operations Administrators (Trusted):** Entities responsible for modifying core economic parameters, rate limits, and system caps behind a time delay.
- **Operations Multisig (Semi-trusted):** Entities managing hard pauses, unpauses, and base operational thresholds.
- **Guardians (Semi-trusted):** Roles capable of triggering time-bound soft pauses during emergencies and tightening specific user rate limits.
- **Oracle Operators (Semi-trusted):** Off-chain actors submitting and executing consensus reports that trigger rebases, withdrawals, and fee distributions.
- **Executor Operations (Semi-trusted):** Entities managing external restaking deposits, validator exits, and consolidations.

- **Housekeeping Operators (Semi-trusted):** Entities responsible for finalizing generic withdrawals, sweeping stranded assets, and fulfilling priority queues.

Trust Assumptions

- **Admin Honesty:** Administrators, multi-sigs, and guardians will not maliciously upgrade contracts, manipulate economic parameters, or indefinitely pause the system to censor users.
- **Oracle Integrity:** Oracle operators will consistently submit accurate and timely reports reflecting exact validator balances, and will manually intervene if race conditions stall the execution pipeline.
- **External Protocol Safety:** Underlying dependencies for staking, restaking, and price feeds operate correctly and cannot be exploited to skew exchange rates or siphon funds.
- **Receiver Compliance:** Users withdrawing native assets utilize correctly configured accounts or smart contracts capable of receiving transfers within restricted gas stipends.
- **MembershipManager Version:** There are no VO NFTs in circulation.

Attack Vectors

- **Rate-Limit Weaponization (Denial of Service):** Exhausting per-address or global rate-limiter capacities via high-volume wash trading to block external liquidations or intentionally halt oracle report execution.
- **Front-Running & MEV Extraction:** Monitoring the mempool to front-run negative rebases with withdrawal requests, griefing remaining liquidity providers, or bundling large deposits with execution calls for risk-free yield.
- **Risk-Free Yield & Fallback Griefing:** Users whose liquidity is not utilized shall not earn rewards.
- **TVL Accounting Desync:** The TVL shall be correctly accounted within the LiquidityPool variables.
- **Finalization Rate Manipulation:** Exploiting the timing discrepancy between withdrawal finalization and actual share burning to manipulate the finalization exchange rate upon claiming.

- **Liquidity Solvency Bypasses:** Solvency shall be preserved. All withdrawals shall be claimable. The pool should not be in a state in which the liquidity is less than the amount reserved for withdrawal.
- **Pause State Deadlocks:** Triggering time-bound soft pauses that inherently conflict with strict non-paused modifiers, inadvertently blocking administrators from executing critical emergency hard pauses.
- **Precision Loss & Asset Leakage:** Leveraging asymmetric ceiling and floor rounding math during withdrawal calculations to systematically force fractional asset leakage to the protocol at the expense of the user.
- **Storage Layout Consistency:** The new implementation contracts shall preserve the existing storage layout by correctly utilizing all previously allocated storage slots. No storage slot shall be overwritten, repurposed incompatibly, or subject to storage collisions, ensuring safe upgradeability and state integrity.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	7	7	6
Informational	26	26	10
Total	33	33	16

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
Low	Low	Low	Low	Medium
	Low	Medium	High	
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	Priority withdrawal claims can be bricked for smart-contract recipients that forward or consume received ETH	Low	Fixed
L-02	Unpublishing an oracle report can permanently block resubmission for the same slot range	Low	Fixed
L-03	Unjustified Reverts in quoteByMarketValue Due to Price Clamping When stETH Trades at a Premium	Low	Fixed
L-04	Operator blacklist bypass via missing msg.sender validation in WithdrawRequestNFT	Low	Fixed
L-05	Invalid requests cannot be seized from blacklisted users	Low	Fixed
L-06	Claims can manipulate finalization rates	Low	Acknowledged
L-07	Invalid requests causing DOS of finalizeWithdrawalsWhenStale	Low	Fixed
I-01	Fast role revocation can be disabled by an immutable misconfigured revoke admin	Informational	Fixed

I-02	EtherFiRedemptionManager permanently strands fractional eETH shares	Informational	Fixed
I-03	cancelWithdraw racing condition	Informational	Acknowledged
I-04	Dead parameters in initialize in multiple contracts	Informational	Fixed
I-05	Oracle Report DoS via invalidateRequest During Reporting Window	Informational	Acknowledged
I-06	Unhandled Stale Withdrawal Finalization Causes DoS of Oracle	Informational	Acknowledged
I-07	PriorityWithdrawalQueue claiming could abuse the mint/burn rate limits	Informational	Acknowledged
I-08	Finalized withdrawals may exceed the liquidity	Informational	Acknowledged
I-09	Users in the priority withdrawal queue can earn rewards, without contributing	Informational	Acknowledged
I-10	The consumed burn amount may be inflated	Informational	Acknowledged
I-11	An attacker could delay any withdrawal request	Informational	Acknowledged
I-12	Oracle reports can become impossible to unpublish after quorum or committee changes	Informational	Acknowledged

I-13	Blacklisting the zero address can cause protocol-wide mint/burn and withdrawal-claim denial of service	Informational	Fixed
I-14	Temporary DOS on eETH minting due to rate limiter bucket creation gap	Informational	Fixed
I-15	EtherFiRestaker setPauseUntilDuration missing from operating config batch	Informational	Fixed
I-16	Pre-upgrade remainder eETH shares in PWQ and WRN not swept before upgrade	Informational	Acknowledged
I-17	ISyncPoolImpl bytecode not verified against local compilation	Informational	Acknowledged
I-18	OZ paused state silently lost across upgrade — contracts unpause regardless of pre-upgrade state	Informational	Acknowledged
I-19	RoleRegistry excluded from verifyAccessControlPreservation checks	Informational	Fixed
I-20	ETH swept from AuctionManager to MembershipManager may become inaccessible post-upgrade	Informational	Acknowledged
I-21	Oracle reports straddling the upgrade window cannot be finalized post-upgrade	Informational	Acknowledged
I-22	Liquifier/RedemptionManager stETH price feed heartbeat mismatch causes stale price reverts	Informational	Fixed

I-23	DMIN_MAX_REQUESTS_TO_FINALIZE_PER_REPO RT = 5000 would consume excessive amount of gas	Informational	Fixed
I-24	EtherfiL1SyncPoolETH.setPauseUntilDuration not seeded in Batch 2 operating config	Informational	Fixed
I-25	Halted claims due to request limit being higher than the burn limit	Informational	Acknowledged
I-26	Invalidation of blacklisted users' priority withdrawal requests permanently locks ETH	Informational	Acknowledged

Low Severity Issues

L-01: Priority withdrawal claims can be bricked for smart-contract recipients that forward or consume received ETH

Severity: Low	Impact: Low	Likelihood: Low
Files: src/withdrawals/PriorityWithdrawalQueue.sol	Status: Fixed	

Description:

`claimWithdraw()` captures `user.balance` before payment and subsequently asserts `user.balance > userEthBefore`. If the recipient is an account-abstraction wallet or splitter contract that automatically forwards, wraps, or utilizes the ETH synchronously, its final balance will not have increased. This assertion causes valid claims to universally revert for such contracts.

Recommendations:

Remove strict final recipient balance assertions; instead verify internal protocol invariant transitions (e.g., escrow decrease) and the success boolean of the raw ETH transfer.

Customer Response:

Fixed in [Pr-466](#)

Fix Review:

Fix confirmed

L-02: Unpublishing an oracle report can permanently block resubmission for the same slot range

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/oracle/EtherFiOracle.sol	Status: Fixed	

Description:

When committee members submit an oracle report, their `lastReportRefSlot` is immediately updated. If the report reaches consensus and is later unpublished via `unpublishReport`, the report consensus is cleared, but the submitting members' `lastReportRefSlot` is not reset. Those members can no longer submit a corrected report for the same slot range. If they are required for quorum, the oracle becomes unable to publish a corrected report. Additionally, unpublishing a report with a zero slot or block underflows the previous variables.

Recommendations:

Consider tracking per-report submitters and resetting their submission state when a report is unpublished, and safely handling the zero slot/block case without underflow.

Customer Response:

Fixed in [Pr-466](#)

Fix Review:

Fix confirmed

L-03: Unjustified Reverts in quoteByMarketValue Due to Price Clamping When stETH Trades at a Premium

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/deposits/Liquifier.sol	Status: Fixed	

Description:

In the `Liquifier` contract, the `quoteByMarketValue` function calculates `_marketValue` by clamping the Curve pool price to a maximum of `_amount` (a 1:1 ratio) to ensure stETH deposits are never credited for more than their underlying ETH value.

However, the deviation check against the Chainlink oracle uses this *clamped* `_marketValue`. If stETH naturally trades at a premium to ETH that exceeds `maxPriceDeviationInBps`, the deviation between the clamped value (1:1) and the true Chainlink price will trigger an `InvalidStEthPrice()` revert.

This results in an unintended Denial of Service (DoS) for stETH deposits during periods where stETH is priced higher than ETH, even when both the oracle and the AMM are reporting accurate prices.

Recommendation:

Calculate the deviation by comparing the Chainlink price feed against the *unclamped* Curve pool price to accurately verify market conditions. Apply the `_min(_amount, ...)` clamping to determine the final `_marketValue` only *after* the deviation check has successfully validated the true market prices.

Customer Response:

Fixed in [Pr-466](#)

Fix Review:

Fix confirmed

L-04: Operator blacklist bypass via missing msg.sender validation in WithdrawRequestNFT

Severity: Low	Impact: Low	Likelihood: Low
Files: src/withdrawals/WithdrawRequestNFT.sol	Status: Fixed	

Description:

In `WithdrawRequestNFT._beforeTokenTransfer`, the internal hook explicitly enforces blacklisting checks on the `from` and `to` addresses, but fundamentally omits checking the `msg.sender` (the approved operator). This inconsistency departs from the more secure logic implemented in `EETH`, `WeETH`, and `MembershipNFT`. Consequently, an approved smart contract or delegated operator that has been officially blacklisted can entirely bypass the restriction and continue transferring un-blacklisted owners' NFTs.

Recommendations:

Include `blacklist.nonBlacklisted(msg.sender);` inside `WithdrawRequestNFT._beforeTokenTransfer` to ensure approved operators are scrutinized against the blacklist, maintaining uniform access control across the protocol.

Customer Response:

Fixed in [Pr-466](#)

Fix Review:

Fix confirmed

L-05: Invalid requests cannot be seized from blacklisted users

Severity: Low	Impact: Low	Likelihood: Low
Files: src/withdrawals/WithdrawRequestNFT.sol	Status: Fixed	

Description:

When a user gets blacklisted, the transfer within `seizeInvalidRequest` is going to revert:

```
function seizeInvalidRequest(uint256 requestId, address recipient) external
onlyUpgradeTimelock {

    if (_requests[requestId].isValid) revert RequestValid();
    if (!_exists(requestId)) revert RequestNotFound();

    _transfer(ownerOf(requestId), recipient, requestId);
...}
function _beforeTokenTransfer(address from, address to, uint256 firstTokenId,
uint256 batchSize) internal override {

    blacklister.nonBlacklisted(from);
    blacklister.nonBlacklisted(to);
}
```

Recommendations:

Consider allowing the `onlyUpgradeTimelock` to seize invalid requests from blacklisted users.

Customer Response:

Fixed in [Pr-466](#)

Fix Review:

Fix confirmed

L-06: Claims can manipulate finalization rates

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/withdrawals/WithdrawRequestNFT.sol	Status: Acknowledged	

Note: *The severity of this issue has been decreased to Low, due to the newly introduced rebase limits. Also the likelihood of negative rebase happening has been estimated to be very low.*

Description:

In `WithdrawRequestNFT::finalizeRequests` the rate is snapshotted:

```
function finalizeRequests(uint256 requestId) external {  
    ...  
    uint256 rate = liquidityPool.amountPerShareCeil();  
    _finalizationRates.push(uint32(requestId), uint224(rate));  
}
```

After the finalization of their request, the user can receive the minimum between the amount they originally requested – `request.amountOfEEth` or the amount that is derived from the finalized rate.

However, the user could claim at a later stage – days or weeks after the finalization. If the rate has rebased negatively after the finalization, the user's shares would temporarily absorb the loss. After they claim, the loss is instantly going to be shifted to the remaining liquidity providers, decreasing the share price.

As a result, user's claims would be able to predictably shift the share price.

Consider the following scenario:

1. Alice and Bob are both liquidity providers in the pool, each having 50 shares. The pool has a total of 100 assets.

2. Bob requests a withdrawal of 30 shares. Bob's request is finalized at a ratio 1:1. However, Bob does not claim his request yet.
3. A negative rebase occurs – now 1 share is worth 0.5 assets. Bob has 30 shares, with a locked ratio of 1:1. While Bob's finalized shares are not going to be affected by the negative rebase.

The state of the pool becomes the following:

- totalSupply = 100 shares
- totalAssets = 50 assets

Now, if Bob uses the redemptionManager, he could instantly withdraw his remaining 20 shares, for 10 assets (assuming no fee) and the other 30 assets from his initial request. However, if he claims, before he redeems his remaining shares, the outcome is going to be the following:

- Bob claims his 30 shares, which have been finalized at a 1:1 ratio. He receives 30 assets. The state in the pool becomes:
 - totalSupply – 70
 - totalAssets – 20.
- After Bob's claim, one share is worth ~0.28 assets each.
- If he withdraws his remaining shares, he is going to receive ~ 5.7 assets, instead of the 10 assets, which he would have withdrawn, if he changed the order of operations.

As a result, any claim after a negative rebase is going to push the loss to the rest of the liquidity providers, creating opportunities for active users.

Recommendations:

Consider burning the shares and reserving the assets upon finalization

Customer Response:

Acknowledged

L-07: Invalid requests causing DOS of finalizeWithdrawalsWhenStale

Severity: Low	Impact: Low	Likelihood: Low
Files: src/oracle/EtherFiAdmin.sol	Status: Fixed	

Description:

In `finalizeWithdrawalsWhenStale`, invalid requests are skipped, but are still counted towards the `maxNumberOfRequestsToFinalizePerReport` cap. If the next chunk of pending IDs are all invalid (or only invalid ones fit in one pass), `finalizedWithdrawalAmount` stays 0 and the function reverts on `NoWithdrawalsToFinalize()`:

```
if (finalizedWithdrawalAmount == 0) revert NoWithdrawalsToFinalize();
```

Recommendations:

Consider a mechanism to handle the invalid requests – for example not reverting on 0 amounts, just so `requestId` can advance

Customer Response:

Fixed in [Pr-481](#)

Fix Review:

Fix confirmed

Informational Severity Issues

I-01: Fast role revocation can be disabled by an immutable misconfigured revoke admin

Files:

src/governance/RoleRegistry.sol

Description:

`RoleRegistry` stores `revokeAdmin` as an immutable constructor argument and does not validate it. If it is set to a placeholder, zero address, or an incorrect proxy address, the `revokeFast()` function will consistently revert with `OnlyRevokeAdmin()`. This renders the dedicated fast-revocation path unavailable during an emergency until an upgrade or redeployment can be completed.

Recommendations:

Validate against `address(0)` in the constructor and ensure the provided address corresponds to the deployed `RevokeAdmin` proxy. Alternatively, store `revokeAdmin` in upgradeable storage with a timelocked setter.

Customer Response:

Fixed in [PR-466](#)

Fix Review:

Fix confirmed

I-02: EtherFiRedemptionManager permanently strands fractional eETH shares

Files:

src/withdrawals/EtherFiRedemptionManager.sol

Description:

When processing instant redemptions and assessing exit fees, `EtherFiRedemptionManager` calculates the fee initially as shares (`feeShareToTreasury`), converts these shares into an equivalent physical ETH amount, and then calls `eETH.transfer()` to send the funds to the treasury. The `eETH` token internally converts this ETH amount back into shares (`sharesToTransfer = liquidityPool.sharesForAmount(amount)`). Due to floor rounding through dual conversions, the shares actually transferred mathematically fall short of the originally derived `feeShareToTreasury`. The residual dust shares are stranded indefinitely inside the Manager contract.

Recommendations:

Perform the treasury transfer using a native share-transfer mechanism to bypass dual-conversion rounding. Alternatively, implement an admin-controlled `sweep` mechanism to recover stranded fractional tokens.

Customer Response:

Fixed in [PR-466](#)

Fix Review:

Fix confirmed

I-03: cancelWithdraw racing condition

Files:

src/withdrawals/PriorityWithdrawalQueue.sol

Description:

`PriorityWithdrawalQueue` allows any address to call `claimWithdraw` on a finalized, matured request. ETH is always sent to `request.user`, not the caller. A third party can front-run a user's `cancelWithdraw` with `claimWithdraw` in the same block. That does not steal funds, but it forces the ETH exit path and can block the user's preferred eETH refund path. There is also a pause asymmetry: cancel is blocked while paused; claim is not.

Recommendations:

Consider not allowing cancellations after the withdrawal is completed.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

I-04: Dead parameters in initialize in multiple contracts

Description:

The following contracts receive inputs in the initialize function, but do not store them:

EtherFiAdmin

LiquidityPool

EETH

WeETH

WithdrawRequestNFT

Liquifier

EtherFiRestaker

AuctionManager

MembershipNFT

Recommendations:

Consider removing the redundant inputs

Customer Response:

Fixed in [PR-466](#)

Fix Review:

Fix confirmed

I-05: Oracle Report DoS via invalidateRequest During Reporting Window

Files:

src/oracle/EtherFiAdmin.sol

Note: *The severity of this issue has been downgraded to Informational, as the protocol's operational assumptions rely on off-chain coordination and trusted actors behaving as intended. Specifically, requests are not expected to be invalidated once consensus has been reached.*

Description:

The `EtherFiAdmin` contract validates oracle reports by summing the amounts of all valid withdrawal requests up to `_report.lastFinalizedWithdrawalRequestId` and strictly comparing this sum against `_report.finalizedWithdrawalAmount`. If an admin calls `invalidateRequest()` on a pending withdrawal request in the `WithdrawRequestNFT` contract after the oracle captures its snapshot but before `executeTasks()` is executed, the invalidated request's amount will be excluded from the on-chain sum. This mismatch causes the strict equality check in `_validateWithdrawals()` to fail, resulting in a Denial of Service (DoS) for the entire oracle report.

Recommendations:

Consider adding softer validation on the withdrawals instead of a strict equation.

Customer Response:

Acknowledged.

We can configure guardian to not invalidate a request once consensus has reached.

Fix Review:

Acknowledged

Invalidating requests after any of the members has submitted a report could cause issues with reaching a quorum. Consider invalidating requests, while their finalization is not yet a part of the oracle report for the particular period.

I-06: Unhandled Stale Withdrawal Finalization Causes DoS of Oracle

Files:

src/oracle/EtherFiAdmin.sol

Note: The severity of this issue has been decreased to informational, as the stale window will be set to 14 days, making a collision with an incoming report highly unlikely.

Description:

The `finalizeWithdrawalsWhenStale` escape hatch allows anyone to finalize pending withdrawals during oracle delays. However, if triggered concurrently with an oracle report containing those same withdrawals, the report is permanently DoS'd. `_validateWithdrawals` verifies the report's `finalizedWithdrawalAmount` by summing requests starting from `lastFinalizedRequestId()`. Since the escape hatch advanced this ID, the sum falls short, causing `executeTasks` to revert. This halts the Oracle entirely, as `shouldSubmitReport` requires the previous report to execute successfully, mandating manual multisig intervention via `unpublishReport`. The new report should be modified to exclude the withdraws that have been executed in order to make it executable

Recommendations:

Document this edge case in runbooks so the multisig quickly calls `unpublishReport` if a race condition occurs. Alternatively, update `_validateWithdrawals` and `_finalizeWithdrawals` to gracefully handle an already-advanced `lastFinalizedRequestId`, allowing the rest of the report (fees, rebases) to execute successfully.

Customer Response:

Acknowledged

"If oracle is inactive for close to the stale duration (which is set to 14 days), then we will wait for `finalizeWithdrawalsWhenStale` to be executable, execute it ourself which puts it under 1 day cooldown, and then restart the oracle service safely and execute task"

Fix Review:

Acknowledged

I-07: PriorityWithdrawalQueue claiming could abuse the mint/burn rate limits

Files:

src/withdrawals/PriorityWithdrawalQueue.sol

Description:

After requests have been fulfilled within the `PriorityWithdrawQueue`, anyone could invoke `claimWithdraw` for requests that belong to other users. In `_claimWithdraw`, the following is invoked:

```
uint256 burnedShares = liquidityPool.withdraw(amountToWithdraw, rate,
request.shareOfEEth);
```

This will attempt to burn shares from the `PriorityWithdrawQueue`:

```
eETH.burnShares(msg.sender, share);
```

An attacker could target a withdrawal request and delay it, by claiming all of the stale requests, which have not been claimed yet.

Recommendations:

Consider acknowledging this issue.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

I-08: Finalized withdrawals may exceed the liquidity

Files:

src/oracle/EtherFiAdmin.sol

Description:

When withdrawals are finalized, `EtherFiAdmin` moves ETH from the liquidity pool into `WithdrawRequestNFT` using each request's original `amountOfEEth` — the nominal eETH amount recorded at request time. Oracle validation (`_validateWithdrawals`) and the stale-oracle path (`finalizeWithdrawalsWhenStale`) both require that the sum of these nominal amounts fits within `liquidityPool.totalValueInLp()`. After a negative rebase from validator slashing, LP liquidity falls while pending requests still carry their pre-slash `amountOfEEth`, so finalization can be rejected even though `WithdrawRequestNFT::finalizeRequests` would snapshot the lower post-slash share rate and claims would pay $\min(\text{amountOfEEth}, \text{shareOfEEth} \times \text{frozenRate})$ — less than the nominal amount. The result is that affected withdrawals may remain unfinalized until LP liquidity recovers enough to cover the full pre-slash nominal total, despite there being sufficient liquidity to honor the actual post-slashing claim amounts.

Recommendations:

If there has been a slashing, consider using the actual amount that is going to be withdrawn, instead of the originally requested amount.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

I-09: Users in the priority withdrawal queue can earn rewards, without contributing

Files:

src/withdrawals/PriorityWithdrawalQueue.sol

Description:

The `PriorityWithdrawalQueue` contract enables users to cancel or claim their withdrawal requests.

For claiming, anyone can initiate a claim on behalf of another user. However, the following transfer may revert the whole claim:

```
(bool ok,) = payable(request.user).call{value: amountToWithdraw}("");
```

As a result, users can have their withdrawal requests stale for a long time. Furthermore, they can cancel their requests after the completion:

```
function cancelWithdraw(
    WithdrawRequest calldata request
)
    external
    nonReentrant
    whenNotPaused
    onlyRequestUser(request.user)
    returns (bytes32 requestId)
{
    ...
    uint256 expectedLpEthDelta = wasFinalized ? uint256(request.amountOfEEth) :
0;
```

Once they cancel, they receive a full refund for their request:

```
uint256 amountForShares = liquidityPool.amountForShare(request.shareOfEEth);
```

A malicious user may deposit a large amount of funds, and create a request. Once their request is claimable, they still earn rewards (assuming they cancel in the future), without having their funds utilized by the protocol. Furthermore, they can withdraw at any point in the future.

However, the `oracleOperations` role may invalidate the request of such users.

Recommendations:

Consider enforcing a cancellation period, after which a request cannot be cancelled.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

I-10: The consumed burn amount may be inflated

Files:

src/core/EETH.sol

src/core/WeETH.sol

Description: When shares are burned, the following bucket limit logic is enforced:

```
uint256 amount = liquidityPool.amountForShare(_share);  
  
    rateLimiter.consumeToken(EETH_BURN_LIMIT_ID, toBucketUnit(amount));
```

However, the amount variable may not accurately reflect the actual burned amount. For example, in `LiquidityPool::withdraw()`, the amount that is withdrawn may not be equal to the `amountForShare(_share)`, as the withdrawal is executed at the finalization rate.

Recommendations:

Due to the negligible impact, consider acknowledging this issue.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

I-11: An attacker could delay any withdrawal request

Description: The `EtherFiAdmin` contract enforces the following limit on task execution:

```
if (_report.lastFinalizedWithdrawalRequestId - lastFinalizedRequestId >
maxNumberOfRequestsToFinalizePerReport)
```

The minimum amount that a user can request directly from the `LiquidityPool` is set using the `minWithdrawAmount` variable. However, there is no minimum amount enforced when a withdrawal is initiated from the `MembershipManager` contract.

An attacker may target a big `withdrawal request` and delay it indefinitely by spamming `withdrawal requests`, filling up the withdrawal queue and exceeding the `maxNumberOfRequestsToFinalizePerReport`.

Additionally, if the `minWithdrawAmount` is set to a low value, such griefing could be performed directly from the `LiquidityPool` contract.

Recommendation: Consider introducing a limit on the `MembershipManager` contract that would be sufficient to prevent abuse of `requestWithdraw`.

Customer's response: Acknowledged

"For Membership NFTs, total number of holders are 1571 which is less than the 3500 value set for `maxNumberOfRequestsToFinalizePerReport`. `minWithdrawAmount` will be set to 0.001 ETH which puts the cost of griefing at 3.5 ETH, since we have the ability to seize all these requests and blacklist the user, this leaves no desirable vector for an attacker."

I-12: Oracle reports can become impossible to unpublish after quorum or committee changes

Description: `unpublishReport()` validates the `_members` array against the current `quorumSize` and the current `committeeMemberStates`:

JavaScript

```
if (_members.length < quorumSize) revert InvalidMembersLength();

CommitteeMemberState storage memberState = committeeMemberStates[_members[i]];
if (memberState.lastReportRefSlot != _report.refSlotTo) revert MemberNotParticipated();
```

However, the report being unpublished may have reached consensus under an older quorum and committee set. If the operating roles change committee membership or increase quorum after a report is published but before it is executed or unpublished, recovery can become impossible.

For example:

- A report reaches consensus with quorum 3.
- The report is later found invalid but has not yet been executed.
- Governance/operations adds committee members or increases quorum to 5, or removes one of the original participants.
- `unpublishReport()` now requires at least 5 currently participating members, but only 3 members participated in the original report, or some participants were removed and their state was deleted.
- The invalid report cannot be unpublished, while `shouldSubmitReport()` blocks new reports because the last published report has not been handled.

This can permanently stall oracle progress until another upgrade or privileged intervention outside the intended unpublish flow.

This is a low likelihood situation, since it requires the admin to not follow the config procedure. Still it is a valid observation that uncovers some of the system limitations.

Recommendation: Store report-level consensus metadata at publication time, including the quorum used and the participating members or member bitmap. `unpublishReport()` should



validate against the quorum and membership snapshot that applied when the report reached consensus

Customer's response: Acknowledged

I-13: Blacklisting the zero address can cause protocol-wide mint/burn and withdrawal-claim denial of service

Description: `Blacklister` allows privileged callers to blacklist any address, including `address(0)`. Several token hooks call `blacklister.nonBlacklisted()` on `from` or `to` without excluding mint/burn zero-address flows.

For example, `WeETH._beforeTokenTransfer()` checks both `from` and `to`, so blacklisting `address(0)` causes wraps and unwraps to revert because mints and burns use the zero address. `WithdrawRequestNFT._beforeTokenTransfer()` also checks the zero address during mints/burns, which can block withdrawal request creation and finalized claims.

This lets a `GUARDIAN_ROLE` holder blacklist `address(0)` for three days via `blacklistUserUntil()`, creating a broad protocol DoS that bypasses more specific pause-role separation, such as the stricter `SUPER_GUARDIAN_ROLE` intended for pausing `eETH/weETH` token flows.

Recommendation: Reject `address(0)` in all blacklist mutation functions.

Customer's response: Fixed in [PR-481](#)

Fix Review: Fix confirmed

I-14: Temporary DOS on eETH minting due to rate limiter bucket creation gap

Description: New eETH implementation call `rateLimiter.consumeToken()` on every minting and burning. The token contracts are upgraded in Batch 1 (Day 10) but the `rate limiter buckets` and consumer registrations are only created in Batch 2 (2-day `OPERATING_TIMELOCK`). In the window between Batch 1 and Batch 2 execution, every eETH minting or burning operation would revert.

Recommendation: Consider setting the bucket in advance before the upgrade

Customer's response: Fixed in [Pr-481](#)

Fix Review: Fix confirmed.

I-15: EtherFiRestaker setPauseUntilDuration missing from operating config batch

Description: `_buildOperatingConfigBatch()` calls `setPauseUntilDuration` for 11 contracts but omits `EtherFiRestaker`. Additionally, there is no `PAUSE_UNTIL_ETHERFI_RESTAKER` constant in `Constants.s.sol`. After Batch 2, `EtherFiRestaker`'s pause duration silently falls back to `MIN_PAUSE_DURATION` (8 hours) rather than the protocol-configured value.

Recommendation: Consider calling `EtherFiRestaker::setPauseUntilDuration()`.

Customer's response: Fixed in [Pr-481](#)

Fix Review: Fix confirmed

I-16: Pre-upgrade remainder eETH shares in PWQ and WRN not swept before upgrade

Description: `PriorityWithdrawalQueue.totalRemainderShares` and `WithdrawRequestNFT.totalRemainderEEthShares` accumulate eETH dust from withdrawal rounding, drainable only via `handleRemainder()` on the OLD impl. Both functions are removed in new implementations and the slots become `__gap`. No pre-upgrade sweep calls exist in the script, mirroring the `AuctionManager` pattern but left unhandled — the eETH shares are permanently stranded post-upgrade.

Recommendation: Consider sweeping the remainders before the upgrade executes

Customer's response: Acknowledged.

""We will manually trigger the handleRemainder workflow before the upgrade""

I-17: IISyncPoolImpl bytecode not verified against local compilation

Description: `verifyDeployedBytecode` rebuilds and compares all new implementations from the local working tree. `IISyncPoolImpl` originates from a separate repository (`WeETH-cross-chain`, PR #77) and is not compiled locally. No bytecode verification is performed for this implementation.

Recommendation: Consider acknowledging this issue if it is intentional.

Customer's response: Acknowledged

I-18: OZ paused state silently lost across upgrade — contracts unpause regardless of pre-upgrade state

Description: Old implementations read `_paused` from OZ linear storage. New `DeprecatedOZPausable` preserves that slot but exposes no getter. New `Pausable` reads from a separate namespaced slot (`keccak256("pausable.storage")`), which starts at zero. Any contract paused before the upgrade appears unpaused post-upgrade via `paused()`. The `verifyAccessControlPreservation` check passes vacuously since production contracts are unpaused, masking the semantics.

Recommendation: Consider migrating the paused state during the upgrade.

Customer's response: Acknowledged

I-19: `RoleRegistry` excluded from `verifyAccessControlPreservation` checks

Description: `ROLE_REGISTRY` is upgraded in Phase B of Batch 1 but is absent from `_upgradedProxies()`. No pre-upgrade snapshot is taken for its `owner` or `paused` state, no post-upgrade assertions are made, and `verifyNotReinitializable` is never called on it.

Recommendation: Consider adding sanity checks for `RoleRegistry` which is an important element of the system.

Customer's response: Fixed in [Pr-481](#)

Fix Review: Fix confirmed.

I-20: ETH swept from AuctionManager to MembershipManager may become inaccessible post-upgrade

Description: Batch 0 flushes `accumulatedRevenue` ETH to `MembershipManager`. `MembershipManager` is then upgraded in Batch 1 (10 days later). If the new `MembershipManager` removes or changes the functions responsible for processing or distributing this ETH balance, the swept funds are stranded. More specifically if the threshold in `rebase()` is not met, the contract would not be able to process the funds sent from `AuctionManager` before the upgrade and they will remain stuck after that, as no ETH handling logic would exist in it.

Recommendation: In case the above is not intentional consider adding a function to sweep and leftover Eth in the contract – now or in a future update.

Customer's response: Acknowledged

Fix Review: Acknowledged

I-21: Oracle reports straddling the upgrade window cannot be finalized post-upgrade

Description: Oracle reports submitted pre-upgrade using old report structure or **hash format** may fail validation in the new **EtherFiAdmin/EtherFiOracle**. Also such reports cannot be **unpublished** due to the change in **report hashing**, which would require a separate upgrade to remediate the situation.

Recommendation: Make sure to carefully handle the oracle reports before the upgrade so that reporting can migrate smoothly into the upgraded protocol.

Customer's response: Acknowledged. *"After the upgrade becomes executable, we wait for the oracle to execute the admin task after which we shut down the oracle service. We then execute the upgrade transaction and spin up a new oracle service compatible with all the changes made in the upgrade. All this takes place in a few hours to ensure time between rebases never exceeds 24 hours."*

Fix Review: Acknowledged

I-22: Liquifier/RedemptionManager stETH price feed heartbeat mismatch causes stale price reverts

Description: The new **Liquifier** and **RedemptionManager** validate the Chainlink **stETH/ETH price feed** against **stalePriceWindow**. The staleness is set to 2 days, while the heartbeat value is 1 day. As a result **stETH deposit** and **redemption** operations may be based on a **stale price feed**.

Recommendation: Consider using a lower value for a staleness period than 2 days.

Customer's response: Fixed in [PR-481](#)

Fix Review: Fix confirmed.

I-23: DMIN_MAX_REQUESTS_TO_FINALIZE_PER_REPORT = 5000 would consume excessive amount of gas

Description: `EtherFiAdmin.executeTasks` validates up to 5,000 withdrawals via a `getRequest` loop (>20M gas in benchmarks), close to the 30M block cap. A maxed-out withdrawal report can fail to execute or leave almost no room for rebase/validator work in the same block.

Recommendation: Consider decreasing the number of withdrawals.

Customer's response: Fixed in [PR-481](#)

Fix Review: Fix confirmed.

I-24: EtherfiL1SyncPoolETH.setPauseUntilDuration not seeded in Batch 2 operating config

Description: Batch 1 upgrades `EtherfiL1SyncPoolETH` (0xD789...) to an impl that adds `PausableUntil` (WeETH-cross-chain PR #77), but `_buildOperatingConfigBatch()` in `transactions.s.sol` never calls `setPauseUntilDuration` on that proxy. Eleven other L1 contracts get a `_pauseDur(...)` call in Batch 2, and `verifyOperatingConfig()` asserts their `pauseUntilDuration()` values — `L1SyncPool` is excluded from both.

Recommendation: Consider setting the pause duration.

Customer's response: Fixed in [Pr-481](#)

Fix Review: Fix confirmed.

I-25: Halted claims due to request limit being higher than the burn limit

Description: Oracle-side withdrawal finalization (`maxNumberOfRequestsToFinalizePerReport` = 5,000 requests/report and `maxFinalizedWithdrawalAmountPerDay` up to 80,000 ETH once seeded) can approve far more exit volume than a single burst of `claims` can `burn` under the global `EETH_BURN` rate limiter (~25,000 ETH bucket capacity in Batch 2). Each `WithdrawRequestNFT` claim calls `LiquidityPool.withdraw` → `eETH.burnShares`, which consumes `EETH_BURN_LIMIT_ID`; once the bucket is exhausted, further `claims` revert with `LimitExceeded` even though the `requests` are already finalized and ETH is escrowed.

Recommendation: Consider aligning the burn limits.

Customer's response: Acknowledged

I-26: Invalidation of blacklisted users' priority withdrawal requests permanently locks ETH

Description: The `PriorityWithdrawalQueue` allows the `OracleOperations` role to invalidate withdrawal requests using `invalidateRequests()`. When a request is invalidated, the contract internally calls `_cancelWithdrawRequest()`, which refunds the locked ETH back to the `LiquidityPool` and attempts to return the original eETH shares to the user via an ERC20 transfer.

The issue arises when a whitelisted solver or user's wallet is compromised and subsequently blacklisted by the `Guardian` via `Blacklister.blacklistUser()`. In the event of a compromise, the incident response procedure dictates blacklisting the attacker to freeze their assets, followed by the `Oracle` invalidating their pending or finalized priority withdrawal requests to recover the locked ETH back to the `LiquidityPool`.

However, the `eETH.transfer()` function internally enforces blacklist checks on both the sender and the recipient. Because `_cancelWithdrawRequest()` attempts to transfer eETH back to `request.user` (who is now blacklisted), the transfer will revert. Consequently, `invalidateRequests()` will unconditionally revert for any blacklisted user.

The protocol cannot recover the ETH while the attacker is blacklisted, and unblacklisting the attacker would allow them to front-run the invalidation by calling `claimWithdraw()` and stealing the funds.

Recommendations: Do not attempt to refund eETH to blacklisted users during request invalidation, or introduce a seizure mechanism similar to `WithdrawRequestNFT.seizeInvalidRequest()` that transfers the request/funds to a protocol-controlled treasury address instead of the original user.

Customer's response: Acknowledged

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.